

Import statement

```
1 from math import pi
2 tau = 2 * pi
```

Assignment statement

Code (left):

Statements and expressions
Red arrow points to next line.
Gray arrow points to the line just executed

Frames (right):

A name is bound to a value
In a frame, there is at most one binding per name

```
1 from operator import mul
2 def square(x):
3     return mul(x, x)
4 square(-2)
```

Built-in function

User-defined function

Global frame

Intrinsic name of function called

Local frame

Formal parameter bound to argument

Return value

Return value is not a binding!

```
1 from operator import mul
2 def square(x):
3     return mul(x, x)
4 square(square(3))
```

A name evaluates to the value bound to that name in the earliest frame of the current environment in which that name is found.

Evaluation rule for call expressions:

1. Evaluate the operator and operand subexpressions.
2. Apply the function that is the value of the operator subexpression to the arguments that are the values of the operand subexpressions.

Applying user-defined functions:

1. Create a new local frame with the same parent as the function that was applied.
2. Bind the arguments to the function's formal parameter names in that frame.
3. Execute the body of the function in the environment beginning at that frame.

Execution rule for def statements:

1. Create a new function value with the specified name, formal parameters, and function body.
2. Its parent is the first frame of the current environment.
3. Bind the name of the function to the function value in the first frame of the current environment.

Execution rule for assignment statements:

1. Evaluate the expression(s) on the right of the equal sign.
2. Simultaneously bind the names on the left to those values, in the first frame of the current environment.

Execution rule for conditional statements:

- Each clause is considered in order.
1. Evaluate the header's expression.
 2. If it is a true value, execute the suite, then skip the remaining clauses in the statement.

Evaluation rule for or expressions:

1. Evaluate the subexpression <left>.
2. If the result is a true value v, then the expression evaluates to v.
3. Otherwise, the expression evaluates to the value of the subexpression <right>.

Evaluation rule for and expressions:

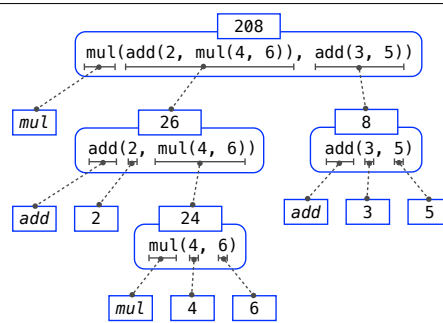
1. Evaluate the subexpression <left>.
2. If the result is a false value v, then the expression evaluates to v.
3. Otherwise, the expression evaluates to the value of the subexpression <right>.

Evaluation rule for not expressions:

1. Evaluate <exp>; The value is True if the result is a false value, and False otherwise.

Execution rule for while statements:

1. Evaluate the header's expression.
2. If it is a true value, execute the (whole) suite, then return to step 1.



Defining:

Formal parameter

Return expression

Def statement

Body (return statement)

```
>>> def square(x):
    return mul(x, x)
```

Call expression: square(2+2)

operator: square

function: func square(x)

operand: 2+2

argument: 4

Calling/Applying:

Argument

Intrinsic name

Return value

```
4 square(x):
    return mul(x, x)
```

```
1 def f(x, y):
2     return g(x)
3
4 def g(a):
5     return a + y
6
7 result = f(1, 2)
```

Global frame

f1: f [parent=Global]

f2: g [parent=f1]

Error

"y" is not found

"y" is not found

- An environment is a sequence of frames
- An environment for a non-nested function (no def within def) consists of one local frame, followed by the global frame

```
1 from operator import mul
2 def square(x):
3     return mul(x, x)
4 square(4)
```

Global frame

f1: square [parent=Global]

mul

square

Return value

16

A call expression and the body of the function being called are evaluated in different environments

```
def fib(n):
    """Compute the nth Fibonacci number, for N >= 1."""
    pred, curr = 0, 1 # Zeroth and first Fibonacci numbers
    k = 1 # curr is the kth Fibonacci number
    while k < n:
        pred, curr = curr, pred + curr
        k = k + 1
    return curr
```

```
def cube(k):
    return pow(k, 3)

def summation(n, term):
    """Sum the first n terms of a sequence.
```

Function of a single argument (not called term)

A formal parameter that will be bound to a function

```
>>> summation(5, cube)
225
"""
total, k = 0, 1
while k <= n:
    total, k = total + term(k), k + 1
return total
```

0 + 1³ + 2³ + 3³ + 4³ + 5³

The cube function is passed as an argument value

The function bound to term gets called here

Pure Functions

```
-2 abs(number):
    2

2, 10 pow(x, y):
    1024
```

Non-Pure Functions

```
-2 print(...):
    None

display "-2"
```

```
1 def strconcat(a, b):
2     print(a + " " + b)
3
4 strconcat("hello", "world")
```

hello world

A and B:
True if A is True and B is True

A or B:
True if A is True or B is True

not A:
True if A is False
False if A is True

```
def abs_value(x):
    1 statement,
    3 clauses,
    3 headers,
    3 suites,
    2 boolean contexts
    if x > 0:
        return x
    elif x == 0:
        return 0
    else:
        return -x
```

Higher-order function: A function that takes a function as an argument value or returns a function as a return value

Nested def statements: Functions defined within other function bodies are bound to names in the local frame


```
Values that are false in a boolean context:
• 0 and 0.0
• False and None
• An empty container, such as '', [], {}, {}
Others are true values.

Membership
>>> digits = [1, 8, 2, 8]
>>> 2 in digits
True
>>> 1828 not in digits
True
>>> [1, 8] in digits
False
>>> 'd' in 'hotdog'
True
>>> 'dog' in 'hotdog'
True
```

Error Types

IndexError	indexing a list past its last element
KeyError	a key isn't in a dictionary
TypeError	an operation on the wrong type: calling a function with wrong number of arguments; indexing or calling None
AttributeError	an attribute is not found for an object
NameError	A name is not bound in the environment

Lambda & Currying: e.g. $f(x, y) \rightarrow f(x)(y)$

```
>>> square = lambda x: x * x # anonymous function
>>> curry2 = lambda f: lambda x: lambda y: f(x, y)
>>> square(4), curry2(add)(3)(4)
(16, 7)
```

Function Inputs, Outputs, & Type Hints

Always ask: what type goes in, what type comes out?

```
def sum_list(s: list[int]) -> int:
    ...
```

Python doesn't check or enforce type hints. They document a function's intended behavior.

list[int]	list of ints	None	no return
dict[str,int]	str → int	A B	an A or a B
tuple[int,str]	(int,str) pair	tuple[()]	()
Link[int]	T = int	LinkedList	Link or ()

Type variables: A name, usually T, is used to indicate that the same type is involved in two or more formal parameters or return values.

```
def pair[T](a: T, b: T) -> list[T]:
    return [a, b]
```

Lists

```
>>> digits = [1, 8, 2, 8]
>>> len(digits)
4
>>> digits[3]
8
>>> digits[-1] # last
8
>>> [2, 7] + digits * 2
[2, 7, 1, 8, 2, 8, 1, 8, 2, 8]
```

Slicing: `s[<start>:<end>]` makes a new list: from `<start>` (included) up to `<end>` (not included). Defaults: `start=0`, `end=len(s)`. `s[:]` and `list(s)` copy the whole list.

```
>>> s = [6, 5, 4, 3, 2, 1, 0]
>>> s[1:3]
[5, 4]
>>> s[3:]
[6, 5, 4]
>>> s[3:1]
[3, 2, 1, 0]
>>> s[-3:] #last three elements
[2, 1, 0]
```

A slice of a list always evaluates to a list (maybe empty) and never causes an IndexError.

Iteration: for Statements

```
for <name> in <expression>:
    <suite>
```

1. Evaluate the header `<expression>`, which must be an iterable value (a list, tuple, range, dict, etc.)

2. For each element in that iterable value, in order:

- Bind `<name>` to that element in the current frame
- Execute the `<suite>`

Unpacking in a for statement:

```
>>> pairs = [[1, 2], [2, 2], [3, 2], [4, 4]]
>>> same_count = 0
>>> for x,y in pairs:
...     if x == y:
...         same_count += 1
>>> same_count
2
```

Annotations:

- `pairs` is a sequence of fixed-length sequences
- A name for each element in a fixed-length sequence

Ranges

```
range(-2, 2) contains ..., -3, -2, -1, 0, 1, 2, 3, ...
• Length: ending value - starting value.
• Element selection: starting value + index.
>>> r = range(-2, 2)
>>> r
range(-2, 2)
>>> len(r)
4
>>> list(range(-2, 2))
[-2, -1, 0, 1]
>>> list(range(4))
[0, 1, 2, 3]
```

Aggregation Functions for an iterable s

```
sum(s, start=0)
max(s, key=None), max(a, b, ..., key=None)
min(s, key=None), min(a, b, ..., key=None)
all(s)
any(s)
len(s)
```

Annotations:

- `total`
- `largest`
- `smallest`
- `all true?`
- `any true?`
- `length`

```
>>> all([False, True])
False
>>> all([])
True
>>> sum([1, 2])
3
>>> sum([1, 2], 3)
6
>>> sum([])
0
>>> sum([1], [2], [])
[1, 2]
>>> any([False, True])
True
>>> any([])
False
>>> max(1, 2)
2
>>> max([1, 2])
2
>>> max([1, -2], key=abs)
-2
>>> len([1, 2, 3])
3
```

Dictionaries

A container of key-value pairs; keys must be unique; iteration gives the keys; `d[k]` errors on a missing key; `d.get(k, default)` returns default if `k` is not a key.

```
>>> words = {'más': 'more', 'otro': 'other', 'agua': 'water'}
>>> len(words)
3
>>> "agua" in words
True
>>> "water" in words
False
>>> "water" in words.values()
True
>>> words["otro"]
'other'
>>> words.get("otro", "hi")
'other'
>>> words.get("hola", "hi")
'hi'
>>> list(words)
['más', 'otro', 'agua']
>>> list(words.values())
['more', 'other', 'water']
```

List & Dictionary Comprehensions

```
[<map exp> for <name> in <iter exp> if <filter exp>]
Short version: [<map exp> for <name> in <iter exp>]
A combined expression that evaluates to a list by:
1. For each item in the iterable value of <iter exp>, bind <name> to that item, then if <filter exp> evaluates to a true value, include the value of <map exp> in the resulting list
2. <name> is not bound in the current frame.
>>> [word for word in words if len(word) == 4]
['otro', 'agua']
>>> [word for word in words]
['más', 'otro', 'agua']
>>> [words[word] for word in words]
['more', 'other', 'water']
>>> {x: x*x for x in range(3,6)} # dict comprehension
{3: 9, 4: 16, 5: 25}
>>> x
NameError
```

Tuples

```
>>> empty = ()
>>> len(empty)
0
>>> print(empty)
()
>>> (2, 3) + (4, 5)
(2, 3, 4, 5)
>>> conditions = ('rain', 'shine')
>>> conditions[0]
'rain'
>>> ra, sh = conditions
>>> sh
'shine'
```

Classes with @dataclass

A class statement creates a new type. When decorated with `@dataclass`, each name that has a type hint becomes an attribute of the instances of this new class. The `__str__` method within the class determines the output when `str` or `print` is called on an instance.

```
@dataclass
class Position:
    "A geographic position."
    lat: float
    lon: float

    def __str__(self):
        return format_pos(self)

def format_pos(p: Position) -> str:
    "Format the position with directional suffixes."
    assert abs(p.lat) <= 90 and abs(p.lon) <= 180, \
        f'p{r} is not on Earth!'
    ... # formats a Position as 33.87° S, 151.21° E

>>> sydney = Position(-33.87, 151.21)
>>> sydney.lat
-33.87
>>> type(sydney) == Position
True
>>> print(sydney)
33.87° S, 151.21° E
```

Recursion

State what the function takes in and should return.

`sum_digits(n):` takes a positive `int` `n`; returns the sum of its digits.

- Base case: an input simple/small enough that the return value can be found without recursion. One digit: return `n`.
- Recursive case: call on a simpler/smaller input and rely on the recursive call's correctness: `sum_digits(all_but_last)` returns the digit sum of all but the last digit; add last.

```
def sum_digits(n: int) -> int:
    "Sum the digits of positive integer n."
    if n < 10:
        return n
    else:
        all_but_last, last = n // 10, n % 10
        return sum_digits(all_but_last) + last

• def header: like any function
• Conditionals: base cases
• Base: no recursive calls
• Recursive: recursive calls
```

Tree Recursion

A recursive case makes more than one recursive call.

```
def fib(n: int) -> int:
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fib(n-2) + fib(n-1)
```

n	fib(n)	n	fib(n)
0	0	5	5
1	1	6	8
2	1	7	13
3	2	8	21
4	3		

Recursive decomposition: find simpler instances of the same problem that complete most of the work.

`count_partitions(n, m)` = ways to write `n` as a sum of parts up to size `m`. For (6, 4), explore two possibilities:

- use at least one 4: what's left is a partition of 2 - `count_partitions(2, 4)`
- don't use any 4: parts up to 3 - `count_partitions(6, 3)`

The last step of the recursive case is to add the two counts to get a total count for (6, 4).

```
def count_partitions(n: int, m: int) -> int:
    if n == 0:
        return 1
    elif n < 0:
        return 0
    elif m == 0:
        return 0
    else:
        with_m = count_partitions(n-m, m)
        without_m = count_partitions(n, m-1)
        return with_m + without_m
```

Linked Lists

A linked list is either empty `()` or a `Link` with a `first` value and the `rest` of the list.

- An empty list is `()`. Test if `s` is empty with `not isinstance(s, Link)` or `s == ()`.
- There's no indexing: to reach item `k`, follow `.rest` `k` times.

`type LinkedList[T] = Link[T] | tuple[()]`

```
@dataclass
class Link[T]:
    first: T
    rest: LinkedList[T] = () # empty list

    def __str__(self):
        return format_link(self)

def format_link(s: Link) -> str:
    """Return a Link s formatted as items within parentheses."""
    string = '(' + str(s.first)
    remaining = s.rest
    while isinstance(remaining, Link):
        string += ' ' + str(remaining.first)
        remaining = remaining.rest
    assert remaining == (), f'{s!r} is not a LinkedList'
    return string + ')'
```

```
>>> end = Link(5)
>>> s = Link(3, Link(4, end))
>>> s.first
3
>>> s.rest.first
4
>>> s.rest.rest.first
5
>>> s.rest.rest.rest == ()
True
>>> print(s)
(3 4 5)
>>> s.rest.rest.rest == ()
True
>>> print(end)
(5)
>>> print(Link(end, s))
((5) 3 4 5)
```

Examples

```
def len_link(s: LinkedList) -> int:
    """Return the length of a linked list.

    >>> len_link(Link(3, Link(4, Link(5))))
    3
    """
    length = 0
    while isinstance(s, Link):
        length += 1
        s = s.rest
    return length

def len_link(s: LinkedList) -> int:
    if not isinstance(s, Link):
        return 0
    return 1 + len_link(s.rest)

def range_link(start: int, end: int) -> LinkedList[int]:
    if start >= end:
        return ()
    else:
        return Link(start, range_link(start + 1, end))
```

Annotations:

- Stops when `s` is `()`
- Base case: `s` is `()`